

# Class 6: R Functions

Mayla (PID: A19136116)

## Table of contents

|                                                              |   |
|--------------------------------------------------------------|---|
| Background . . . . .                                         | 1 |
| A first function . . . . .                                   | 1 |
| A generate_dna() function . . . . .                          | 3 |
| Generate random protein sequence of length 6 to 13 . . . . . | 6 |
| Are our peptides “unique in nature”? . . . . .               | 8 |
| Connecting our findings to immunology . . . . .              | 8 |

## Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function).

## A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either “**required**” or “**optional**”. The latter have fall-back default values that will be used if the user does not specify them.

Q1. Your first version of the function should add two input numbers together. For example, `add(4, 7)` should return 11.

```
add <- function(x, y=0) {  
  x + y  
}
```



Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

**Q1b.** For your second version, adapt your first function so that it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`.

```
add <- function(x, y=0) {  
  sum(x + y)  
}  
add(4, 7)
```

```
[1] 11
```

```
add(c(4, 7, 10))
```

```
[1] 21
```

**Q1c.** Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, 4)` should return 10. Hint: explore the `...` (dots) argument or the base R function `sum()`.

```
add <- function(x, y=0, z=0) {  
  sum(x, y, z)  
}
```

```
add(10, 100)
```

```
[1] 110
```

```
ans <- add (1, 2, 3)
ans
```

```
[1] 6
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

```
add <- function(x, y = 0, z=0) {
  return(sum(x, y, z))
  cat("Is it break time yet?\n")
}
```

## A generate\_dna() function

A useful function here is the “base R” **sample()** function:

```
sample(1:5, size=3)
```

```
[1] 4 3 2
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(1:5, size=60, replace = TRUE)
```

```
[1] 3 2 1 2 5 3 3 4 1 5 2 5 2 3 3 3 4 2 1 1 3 4 4 2 2 4 4 2 4 4 1 5 3 4 4 4 3 3
[39] 4 5 3 1 5 5 2 5 1 1 3 1 5 2 4 2 3 1 3 4 5 1
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(x=c("A", "C", "G", "T"), size= 10, replace = TRUE)
```

```
[1] "A" "C" "T" "A" "A" "C" "T" "A" "A" "A"
```

**Q2a.** Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return "A", "T", "T", "G", "A", "C". [1 pt]

```
generate_dna <- function(len=10) {
  sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
}
```

```
generate_dna()
```

```
[1] "C" "C" "C" "G" "A" "G" "A" "T" "C" "G"
```

```
generate_dna(len=100)
```

```
[1] "A" "C" "C" "A" "A" "G" "C" "A" "C" "A" "T" "G" "A" "C" "G" "G" "G" "T"
[19] "A" "A" "G" "T" "C" "A" "C" "C" "C" "A" "G" "T" "A" "G" "T" "A" "A" "A"
[37] "T" "G" "A" "G" "G" "A" "A" "A" "A" "T" "A" "C" "A" "C" "G" "G" "G" "A"
[55] "C" "C" "T" "T" "C" "C" "T" "G" "A" "T" "C" "G" "A" "C" "C" "C" "C" "T"
[73] "T" "A" "A" "A" "C" "C" "C" "G" "A" "G" "T" "T" "A" "C" "A" "G" "G" "G"
[91] "C" "T" "C" "A" "A" "G" "T" "C" "T" "T"
```

**Q2b.** Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example "AAGGCTG". [1 pt]

```
generate_dna <- function(len=10, single.element=TRUE) {

  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
  #cat("Hello...")

  if(single.element) {
    # cat("is it me you are looking for...")
    ans <- paste(ans, collapse = "")
  }
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`.



```
generate_dna()
```

```
[1] "TACGATAACA"
```

```
generate_dna(single.element = FALSE)
```

```
[1] "C" "T" "T" "G" "G" "A" "C" "T" "A" "C"
```

```
paste(c("A", "C", "G"), collapse = "")
```

```
[1] "ACG"
```

**Q2c.** Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
>len9  
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello  
  there
```

```
#\n means new line  
# \t means tab
```

```
generate_dna <- function(len=10, single.element=TRUE) {  
  
  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)  
  
  if(single.element) {  
    ans <- paste(ans, collapse = "")  
  }  
  
  ##Format as FASTA with an ID line  
  ##  
  cat(paste(">len", len, "\n", sep="") )  
}
```



```

cat(ans)
cat("\n")
##

return(ans)
}

```

```
x <- generate_dna(9)
```

```

>len9
CCAAGGTAA

```

##Write a `generate_protein()` function

**Q3.** Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```

generate_protein <- function(len) {
  aa <- c("A", "R", "G", "L", "M", "F", "W",
         "K", "Q", "S", "P", "V", "I", "C",
         "Y", "H", "R", "N", "D", "T")

  aa <- sample(x=aa, size=len, replace=TRUE)
  paste(aa, collapse = "")
}

```

```
generate_protein(5)
```

```
[1] "TFQCF"
```

### Generate random protein sequence of length 6 to 13

**Q4.** Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for(l in 6:13) {  
  cat(">", l, "\n", sep="")  
  cat( generate_protein(l), "\n" )  
}
```

```
>6  
FKWGIP  
>7  
YRTVTRG  
>8  
FFHWQPHN  
>9  
QSCIWVYDI  
>10  
RMRHKTRVTK  
>11  
FGKADMQIPPW  
>12  
DWWNMVVDLTDH  
>13  
HLMGTTSPGDANR
```

```
generate_protein(6)
```

```
[1] "ISSQGC"
```

```
generate_protein(7)
```

```
[1] "WGFGVCQ"
```

```
generate_protein(8)
```

```
[1] "VGRWPRYP"
```

```
generate_protein(9)
```

```
[1] "MTRMVGAAHS"
```

## Are our peptides “unique in nature”?

**Q5** Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

| Length (aa) | % identity | % coverage | Unique? (Y/N) |
|-------------|------------|------------|---------------|
| 6           | 100        | 100        | N             |
| 7           | 100        | 100        | N             |
| 8           | 100        | 88         | Y             |
| 9           | 100        | 89         | Y             |
| 10          | 64.29      | 90         | Y             |
| 11          | 80         | 91         | Y             |
| 12          | 83.33      | 92         | Y             |
| 13          | 100        | 62         | Y             |

**Q5b.** Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space ( $20^L$  for a peptide of length  $L$ ) and to the size of the known protein universe.

Very short random peptides are almost always found in the nr database because the number of possible sequences is small ( $20^L$ ), so many organisms will share identical short sequences by chance. As peptide length increases, the number of possible sequences grows exponentially, making it much less likely that a random sequence already exists in the database. Since the known protein universe is finite, longer random peptides are more likely to be unique.

## Connecting our findings to immunology

**Q6:** In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Based on my Q5 results, peptides begin to appear “unique in nature” around lengths of about 8–9 amino acids, suggesting that MHC class II molecules likely bind peptides at least this long. This makes biological sense because very short peptides are extremely common due to the small sequence space ( $20^L$ ), so they are likely to match many proteins, including the body’s own. If the immune system presented very short peptides, it would struggle to distinguish self from non-self and could trigger inappropriate immune responses. By requiring a minimum peptide length, MHC class II ensures that presented peptides are more specific and less likely to occur by chance. This increases the accuracy of T-cell recognition and helps prevent autoimmunity.